

AWS Python Deployment

Session 0 — Expectations and Recap

Cletus Nehinlalei Mangu · AmaliTech Python Chapter

May 2026

Contents

Welcome

Recap of Core AWS Concepts

The Two Deployment Models

AWS Services We Will Use

The Python Application We Will Deploy

Concepts That Matter for Both Sessions

What to Prepare Before Each Session

How the Sessions Will Run

Quick Glossary of Abbreviations

Final Thoughts Before We Begin

Welcome

This is a two-session series on deploying Python applications to Amazon Web Services (AWS). This first document — **Session 0** — covers the concepts you need to understand before the hands-on sessions begin. It is intentionally short. The depth happens in the demos.

What we will cover across the series:

- **Session 1 — Server-Based Deployment.** Deploying a Python web application onto Amazon EC2 (Elastic Compute Cloud) instances, with an Application Load Balancer and Auto Scaling Group in front, and automated deployments using AWS CodeDeploy.
- **Session 2 — Serverless Deployment.** Deploying the same kind of Python application using AWS Lambda, Amazon API Gateway, Amazon S3 (Simple Storage Service), and Amazon DynamoDB — without provisioning any servers.

Plus, in both Session 1 and Session 2 you will find optional advanced sections covering:

- Continuous Integration / Continuous Deployment (CI/CD) using GitHub Actions, with Terraform for infrastructure and AWS CodeDeploy for application rollout.
- Alternative deployment options — AWS Elastic Beanstalk, AWS Lightsail, AWS App Runner — that wrap the core services into simpler experiences.
- A brief introduction to microservices deployments on AWS Elastic Container Service (ECS) Fargate and on multi-Lambda architectures with Amazon EventBridge.

These optional sections are clearly marked. During the live sessions we will only cover them if time permits, but they are in the document so you can study and run them on your own.

What you should already know:

- The AWS Cloud Practitioner level fundamentals (what EC2, S3, Lambda, and IAM are conceptually).
- Basic Python — you can write a Flask or FastAPI application.
- Comfort with the Linux command line.

What you'll be able to do after both sessions:

- Take any Python web application from your laptop and deploy it to AWS in either the server-based or the serverless model.
 - Choose between the two models — and among nine deployment options in total — based on the workload.
 - Set up automated CI/CD deployments so that a `git push` to `main` builds, tests, and ships your application without manual steps.
-

Recap of Core AWS Concepts

A 5-minute refresher on terms we'll use repeatedly.

AWS Regions and Availability Zones

- A **Region** is a geographic area containing AWS data centers — for example, `eu-west-1` is Ireland, `us-east-1` is Northern Virginia, `af-south-1` is Cape Town.
- An **Availability Zone (AZ)** is one isolated data center within a Region. Each Region has 2-6 AZs.
- Resources are **Region-scoped**: an EC2 instance in `eu-west-1` does not exist in any other Region.

For our demos: we will use `eu-west-1` (Ireland) — best latency from Ghana with full service availability.

Identity and Access Management (IAM)

IAM controls **who can do what** in your AWS account.

Concept	What it is
User	A human or system with long-lived credentials (password, access keys).
Role	An identity assumed temporarily by an AWS service or user. No long-lived credentials.
Policy	A JSON document listing allowed or denied actions on specific resources.
Trust Policy	Says who can assume a role (e.g., "Lambda service can assume this role").
Permissions Policy	Says what the role can do once assumed.

Why roles matter: roles let AWS services (like EC2 and Lambda) talk to other services (like S3) without us putting credentials in code. We will use roles in every demo.

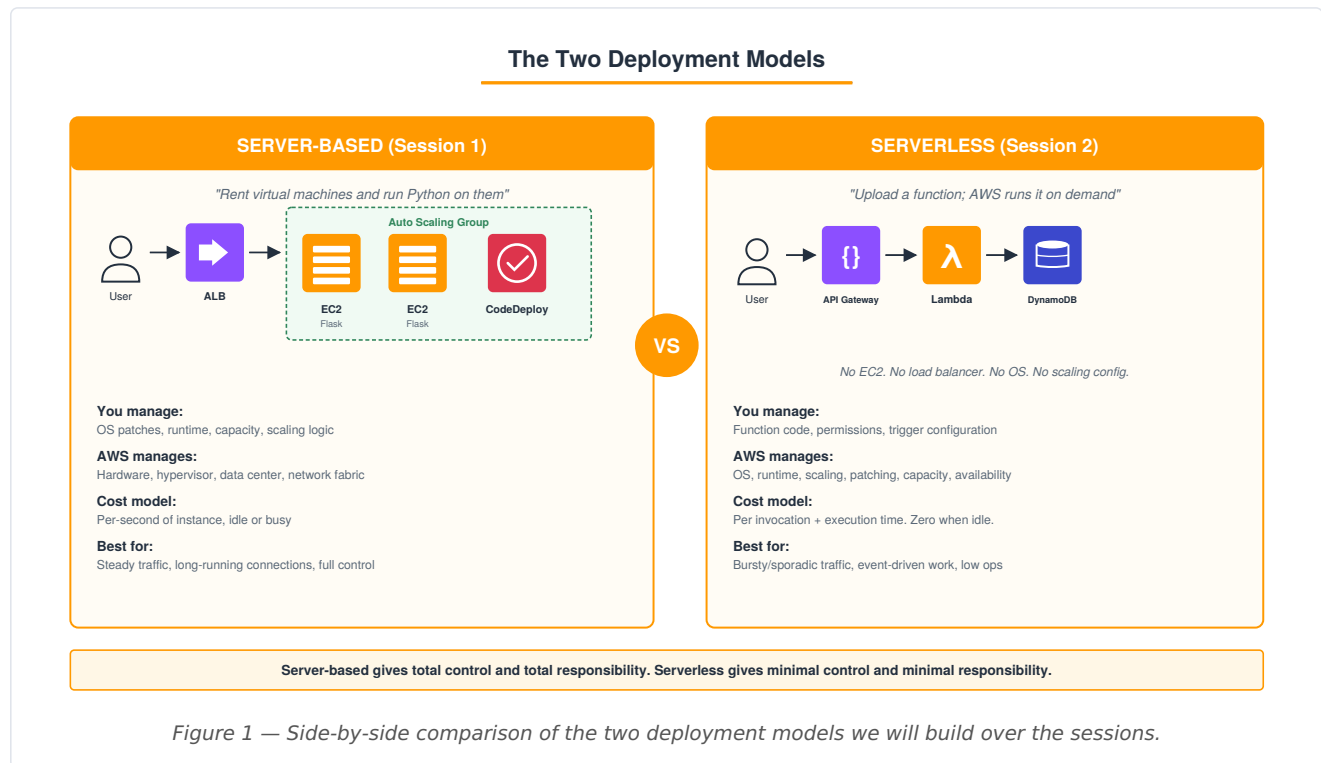
Virtual Private Cloud (VPC)

A VPC is your private network inside AWS. Every AWS account has a default VPC in each Region with public subnets ready to use — we will use those.

Key terms: - **Subnet** — a slice of the VPC tied to one Availability Zone. - **Security Group** — a stateful firewall attached to a resource (default: deny all incoming traffic). - **Internet Gateway** — the door to the public internet.

The Two Deployment Models

This is the single most important framing for both sessions.



Server-based deployment (Session 1)

You rent a virtual machine (an EC2 instance), install Python on it, copy your code there, and run it as a long-lived process. Traffic reaches it via a load balancer. You scale by adding more instances.

You manage: the operating system, the Python runtime, the process supervisor, capacity planning, security patches, deployment automation.

AWS manages: the physical hardware, the hypervisor, the data center.

Cost model: you pay for the instances by the second, whether they are busy or idle.

Serverless deployment (Session 2)

You upload a Python function. AWS runs it on demand. There is no server to log into, no operating system to patch, no scaling to configure.

You manage: the function code, the permissions, the trigger.

AWS manages: the operating system, the runtime, scaling, patching, capacity, high availability.

Cost model: you pay per function invocation and per millisecond of execution time. When no one uses the system, you pay zero.

The decision in one sentence

Server-based gives you total control and total responsibility. Serverless gives you minimal control and minimal responsibility. Every other AWS deployment option lives somewhere on that spectrum.

Neither is universally better. The right choice depends on the workload.

Quick decision framework

Choose server-based when: - Traffic is steady and high enough to run instances efficiently 24/7. - You need custom system libraries or kernel tuning. - A single request can take longer than 15 minutes. - The workload requires long-lived network connections.

Choose serverless when: - Traffic is bursty, sporadic, or unknown. - The work is event-driven (file arrived, message received, schedule fired). - You want zero operational overhead. - You want to pay nothing when idle.

Most real systems use both. A typical pattern: serverless for background jobs and event handlers, server-based for steady-traffic web tiers.

AWS Services We Will Use

A short reference for the services in each session.

Session 1 — Server-Based Services

Service	Full Name	Purpose
EC2	Elastic Compute Cloud	Virtual machines that run your Python application.
EBS	Elastic Block Store	Disk storage attached to EC2 instances.
ALB	Application Load Balancer	Distributes incoming HTTP traffic across multiple instances.
ASG	Auto Scaling Group	Maintains a healthy number of instances; replaces failed ones.
Launch Template	—	Reusable blueprint describing how to launch an instance.
Security Group	—	Virtual firewall around instances and load balancers.
CodeDeploy	AWS CodeDeploy	Automates deployment of new application versions to EC2 instances.
CloudWatch Logs	—	Centralised logging for your application.
IAM	Identity and Access Management	Controls what the EC2 instances and CodeDeploy can do.

Session 2 — Serverless Services

Service	Full Name	Purpose
Lambda	AWS Lambda	Runs Python functions on demand. No servers to manage.
API Gateway	Amazon API Gateway	Turns Lambda functions into HTTPS endpoints.
S3	Simple Storage Service	Object storage. Can trigger Lambda on file uploads.
DynamoDB	Amazon DynamoDB	Serverless NoSQL database.
CloudWatch Logs	—	Centralised logging for your functions.
IAM	Identity and Access Management	Controls what each Lambda function can do.

Optional Advanced Sections — Additional Services and Tools

These appear in the optional advanced sections of both Session 1 and Session 2.

Service / Tool	Purpose
GitHub Actions	Continuous Integration / Continuous Deployment (CI/CD) runner on GitHub.
Terraform	Cloud-agnostic Infrastructure as Code (IaC) tool.
AWS SAM	Serverless Application Model. CloudFormation extension for Lambda + API Gateway.
AWS Elastic Beanstalk	Managed wrapper around EC2 + ALB + ASG for simple deployments.
AWS Lightsail	Flat-rate, simplified virtual machine offering.
AWS App Runner	"Serverless" experience for containerised applications.
AWS Elastic Container Service (ECS) Fargate	Run containers without managing servers.
Amazon Elastic Container Registry (ECR)	Private Docker image registry.
Amazon EventBridge	Event bus for routing events between AWS services and microservices.

The Python Application We Will Deploy

To keep both sessions parallel and comparable, we use a small **Flask** web application in Session 1 and a similar idea expressed as **Lambda functions** in Session 2.

Session 1 application (Flask)

A simple Flask application with three endpoints:

```
from flask import Flask, jsonify, request
import socket, datetime

app = Flask(__name__)

@app.route("/")
def home():
    return jsonify(
        message="Hello from EC2 + Python!",
        host=socket.gethostname(),
        time=datetime.datetime.utcnow().isoformat()
    )

@app.route("/echo", methods=["POST"])
def echo():
    return jsonify(you_sent=request.get_json())

@app.route("/health")
def health():
    return "ok", 200
```

Why the `/health` endpoint? The Application Load Balancer calls it every 15 seconds to verify the instance is still serving traffic. If it stops returning HTTP 200, the load balancer stops sending traffic there.

Session 2 application (Lambda)

The same idea expressed as Lambda handlers. Each function takes `event` and `context` parameters and returns a response dict. For example, a simple handler:

```
def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps({"message": "Hello from Lambda!"})
    }
```

We will build three Lambda-based systems in Session 2: a web Application Programming Interface (API), an event-driven image processor, and a full Create-Read-Update-Delete (CRUD) Representational State Transfer (REST) API with a database.

Concepts That Matter for Both Sessions

A handful of ideas come up in both sessions. Understand them once now.

The runtime environment must match the build environment

This is the single most common cause of "it works on my laptop, but breaks on AWS." Some Python packages contain C code that must be compiled for the target operating system and processor architecture.

- A package built on **macOS** will not run on **Linux**.
- A package built for **x86_64** will not run on **ARM64**.
- A package built for **Python 3.11** will not run on **Python 3.12**.

Pure-Python packages like Flask, requests, and boto3 work everywhere.

C-extension packages like Pillow, NumPy, pandas, and pycpg2 require care:

- **Server-based:** install the packages on the EC2 instance directly (using its package manager), not on your laptop.
- **Serverless:** build the packages inside a Docker container that matches Lambda's runtime, or use a published Lambda Layer.

We will demonstrate both approaches in their respective sessions.

CloudWatch Logs is where the truth lives

Both EC2 instances and Lambda functions write their logs to **Amazon CloudWatch Logs**. When something fails, this is the first place to look.

- For EC2, you need to install the CloudWatch agent or rely on the system journal (`journalctl`).
- For Lambda, CloudWatch logging is automatic — every `print()` and every `logging` call ends up there.

The six stages of any deployment

Every deployment system, anywhere, follows the same six stages. Knowing which stage failed tells you where to look.

#	Stage	What happens
1	Source	Your code lives in version control (Git).
2	Build	Source is turned into a deployable artifact (zip file, Docker image).
3	Package	Artifact is bundled with metadata describing how to run it.
4	Distribute	Artifact is uploaded to where the target can retrieve it (S3, ECR).
5	Run	Old version stops; new version starts.
6	Verify	Health check confirms the new version is working.

Server-based and serverless implement all six. Only the details differ.

Cost discipline

Every AWS resource has a cost — including idle ones. Two non-negotiable habits:

- **Set a billing alarm** in your AWS account at a threshold appropriate for the workload (we will use \$20 for training).
- **Tear down resources after each session.** We will run the cleanup commands together at the end of each session.

The biggest mistake new AWS users make is leaving resources running by accident. A `t3.micro` instance left running for a month costs roughly \$7.50. A NAT Gateway left running for a month costs roughly \$32. A `c5.4xlarge` left running for a month costs roughly \$500.

What to Prepare Before Each Session

To follow along comfortably, attendees should have these ready before each session.

Software

- **AWS Command Line Interface (CLI) version 2.** Install it on your laptop:
 - **Ubuntu / Linux:** `sudo apt install awscli` or follow the official installer.
 - **macOS:** `brew install awscli`.
 - **Windows:** download the Microsoft Installer (MSI) from the AWS CLI documentation.
- **Python 3.10 or newer** with `pip` available.
- **A code editor** — VS Code or anything similar.
- `curl` and `jq` for testing API responses (`sudo apt install curl jq` on Ubuntu).
- For Session 2 only: **Docker Desktop or Docker Engine**, used to build Lambda-compatible packages.

Optional — only if you plan to run the advanced CI/CD or alternative-deployment sections:

- **Terraform** ≥ 1.6 (`brew install terraform` or download from `terraform.io`).
- **AWS Serverless Application Model (SAM) CLI** (`brew install aws-sam-cli`).
- **AWS Elastic Beanstalk (EB) CLI** (`pip install awsebcli`).
- A **GitHub account** with a repository for the CI/CD demos.

AWS Account

- Active AWS account with billing alarm configured.
- An IAM user (not the root user) with sufficient permissions. For training, attach the AWS managed policies: `AmazonEC2FullAccess`, `AWSLambda_FullAccess`, `AmazonS3FullAccess`, `AmazonDynamoDBFullAccess`, `IAMFullAccess`, `AWSCodeDeployFullAccess`, `AmazonAPIGatewayAdministrator`, `CloudWatchFullAccess`.
- AWS CLI configured: run `aws configure` and provide the access key, secret key, and default region (`eu-west-1`).

Verify your setup

Run these three commands. If they all succeed, you are ready:

```
aws --version
aws sts get-caller-identity
aws configure list
```

The first should show AWS CLI version 2. The second should show your account ID and IAM user. The third should show your configured region.

How the Sessions Will Run

Each session lasts **45 to 60 minutes** and follows the same structure:

Phase	Time	Content
Concepts	10 min	Architecture overview, services in scope, key gotchas.
Demo 1	12 min	The simplest possible working version.
Demo 2	15 min	Production-shaped version with HA and scaling.
Demo 3	15 min	Automation and advanced patterns.
Cleanup	5–8 min	Joint walk-through to tear down every resource.
<i>Optional advanced sections</i>	<i>self-paced</i>	<i>CI/CD, Beanstalk/Lightsail/App Runner, microservices.</i>

You can follow along live, or watch and run the demos later on your own. Each session document includes every command in copy-paste form.

On the optional advanced sections: if we have time at the end of a session, we will demo one of them; otherwise they are fully documented for you to run on your own afterward.

A note on cleanup. At the end of each session, we will run the teardown commands together. This is non-negotiable — it prevents surprise bills. We will verify zero resources are running before closing.

Quick Glossary of Abbreviations

Abbreviation	Full term
ALB	Application Load Balancer
AMI	Amazon Machine Image
API	Application Programming Interface
ARN	Amazon Resource Name
ASG	Auto Scaling Group
AZ	Availability Zone
CDK	Cloud Development Kit
CI/CD	Continuous Integration / Continuous Deployment
CLI	Command Line Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DNS	Domain Name System
EBS	Elastic Block Store
EC2	Elastic Compute Cloud
ECR	Elastic Container Registry
ECS	Elastic Container Service
HA	High Availability
HTTP / HTTPS	Hypertext Transfer Protocol / Secure
IaC	Infrastructure as Code
IAM	Identity and Access Management
JSON	JavaScript Object Notation
MFA	Multi-Factor Authentication
NAT	Network Address Translation
OIDC	OpenID Connect
REST	Representational State Transfer
S3	Simple Storage Service
SAM	Serverless Application Model
SDK	Software Development Kit
SG	Security Group
SQS / SNS	Simple Queue / Notification Service
SSH	Secure Shell
TLS	Transport Layer Security

URL	Uniform Resource Locator
VPC	Virtual Private Cloud
WSGI	Web Server Gateway Interface
YAML	YAML Ain't Markup Language

Final Thoughts Before We Begin

These two sessions are not about memorising commands. They are about understanding **how to think about deploying Python on AWS** — what choices you have, what trade-offs each one involves, and how to debug them when they break.

The commands themselves are in these documents. You can copy them. The thinking behind them is what you take away.

Two more things:

- **Ask questions during the demos.** A demo is not a performance. It is a conversation.
- **Run the demos yourself afterwards.** Reading is not the same as doing. The fingertips remember what the eyes forget.

See you in Session 1.